

Status of the Gunter & Scott 1990 Development in Lean

Measured on main at 5a1d8bc. Build 1372 jobs, 0 errors, 0 warnings. 124 modules, 46,097 lines, 2,119 theorems.

1. Extractions — what the paper contains

#	Kind	Count
1	Definitions	11
2	Numbered theorems	16
3	Numbered lemmas	13
4	Numbered propositions	1
5	— numbered results, total	30
6	— the conjuncts those 30 decompose into	93
7	Prose statements	146
8	Examples	1
9	Figures	4

Every count is an extraction from the PDF’s text layer, not an estimate. Results 1–30 are contiguous, no gap and no repeat. Definitions are written Definition: with **no number** — 11 of them, at §2.3, 3.1, 3.2 (×2), 5.2 (×2), 5.3, 6.1 (×2), 7.1, 7.3. The paper labels exactly one Example: (§3.1, strictification); Figure 1 is captioned “Examples of cpo’s” and is counted as a figure.

The extraction must be ligature-aware: the text layer drops fi, so Definition reads De nition and bifinite reads bi nite. A grep for Definition returns 0 and is wrong.

2. Represented in Lean

#	Kind	Represented	Of	Not stated
1	Definitions	11	11	0
2	Theorems	16	16	0
3	Lemmas	13	13	0
4	Propositions	1	1	0
5	Prose statements	86	146	60
6	Examples	1	1	0

All 30 numbered results have been stated in Lean since r0048. The 60 unstated prose statements are concentrated rather than spread: 13 of the 26 unstated *properties* are §7 alone.

3. Corrected — a statement that could not be transcribed as printed

Cause on the left, **what we did about it on the right**. Every one has a corrected statement in Lean; the last two columns say whether that corrected statement is proven.

#	Kind	Printing	Reading	Semantic	Corrected	— proven	— unproven
1	Definitions	1	0	1	2	2	0
2	Theorems	1	1	2	4	2	2
3	Lemmas	1	2	0	3	2	1
4	Propositions	0	0	0	0	0	0
5	— total	3	3	3	9	6	3
6	Prose statements	—	—	—	—	—	not classified
7	Examples	0	0	0	0	0	0

Nothing is left uncorrected. All 9 have a corrected statement in Lean, and 6 of the 9 are proven in that form. The 3 unproven ones are exactly the 3 unproven results in table 4 — no correction is waiting on a decision, only on a proof.

Column meanings, and every row behind the counts:

Printing error — the printed page is unreadable or garbled; the mathematics is right.

#	Where	What	Corrected to	Proven
1	Lemma 9	the PDF drops every $\otimes$ and every $\perp$, so which operators the laws range over is unreadable	six laws recovered, <code>lem9_1...lem9_6</code> ; items 3 and 5 also carry kernel-checked negations of the printed forms	yes
2	Theorem 14	the list of characterizations is garbled	<code>thm14</code> , both directions	yes
3	§7.4’s worked example	reverses the paper’s own definition — printed $b \vdash a$ where both papers give $a \vdash b$	read in the paper’s own direction; the paper’s element counts 1, 2, 5, 20 select this reading over the rival	yes

Reading error — *ours*. The paper is right; we transcribed something it does not say.

#	Where	What	Corrected to	Proven
1	Theorem 29, sentence 2	we dropped the word “domain” from “any bifinite domain”, which makes the sentence false	<code>Thm29SecondAtDomains</code>	no
2	Lemma 28	we quantified over all U ; the paper states it over its own U	<code>PRep.Lemma28AtU</code> , all nine conjuncts, no hypotheses	yes
3	Lemma 30	same shape — we wrote a universal closure the paper does not claim	<code>Lemma30AtV</code> , now at arity 1	no

Semantic error — the mathematics is wrong as stated.

#	Where	Whose	What	Corrected to	Proven
1	Theorem 26	the paper’s	false for any signature with two or more 0-ary slots, including the paper’s own worked $(2, 0, 0, 0, 0, 0)$	<code>thm26</code> with $hs : \forall i, 0 < s\ i$ — the added binder is the repair	yes
2	Theorem 7	ours	our guard tested compactness where the sentence is about boundedness; on the paper’s own index the join always exists, so it tests neither	<code>StepFunctionsDecidable</code> restated over <code>IsStepEnumeration</code> , existentially quantified, naming no guard	no
3	§7.4’s order relation	the paper’s	the printed relation is not reflexive — $b = (\perp, \emptyset)$ fails $b \vdash b$. It is the strict part, and the order is its reflexive closure, which is the identification §7.4 then performs by hand	reflexive closure taken; <code>Colimit.V</code> built on it, with <code>isoPlus : V $\approx_0$ Plus V</code>	yes

Lemma 9’s two misprinted items (3 and 5) are also individually false as printed; they are counted once under Lemma 9’s printing error, since a single unreadable page produced both.

4. Proof state

#	Kind	Proven	Unproven	Of
1	Theorems	14	2	16
2	Lemmas	12	1	13
3	Propositions	1	0	1
4	— numbered results	27	3	30

#	Kind	Proven	Unproven	Of
5	Prose statements	70	16	86 stated
6	Examples	1	0	1

Definitions are not in this table: a definition is not proven or unproven. **25 structures and classes, 0 never instantiated.**

The three unproven numbered results, and what each still needs:

#	Result	State
1	Theorem 7	sentences 1 and 2 proven; the recursive forms are not
2	Theorem 29	sentence 1 proven; sentence 2 is not
3	Lemma 30	stated at V; arity 1 — one named input left

All three reduce to two statements, and nothing else is outstanding:

#	The unproven statement	Blocks
1	RecursiveNormal for $K(D \rightarrow E)$ — decide whether two basis elements are bounded, and compute where their join is	Theorem 7
2	Thm29Normal at infinite $K(E)$ — realize the type over the tower's image, not the η -image. Finite bases are proven	Theorems 29, Lemma 30

Neither is unsolved mathematics: Gunter & Scott proved both in 1990.

5. Proof work state

#	Kind	Started, left with a sorry
1	Definitions	0
2	Theorems	0
3	Lemmas	0
4	Propositions	0
5	Prose statements	0
6	Examples	0

sorry is 0 across the package, and has been since r0042. So is axiom, and so is any constant naming sorryAx.

That number does not mean everything is proven, and the difference matters when reading table 4. A sorry is a hole in a proof somebody *started*. The three unproven results were not started and left open — each is carried as an explicit hypothesis, so a theorem that needs one takes it as an argument. Nothing is asserted that is not proven, and no sorryAx propagates; but no sorry count, here or anywhere, can see this work.

Reproducing

```

scripts/counts.sh           # modules, lines, theorems, sorry
scripts/numbered-status.sh  # declarations per numbered result
scripts/a6-env-scan.sh <out> # then a6-summarize.py
scripts/a8-claim-check.sh   # prose staleness against the environment
scripts/compile.sh -r <round> # build

```